

CS 12 Sample 2

Department of Computer Science
College of Engineering
University of the Philippines Diliman

1 Instructions

1. READ **ALL** OF THESE INSTRUCTIONS **VERY VERY VERY CAREFULLY!!!** 😊

- You may lose points for not following any of the instructions.
2. Answer problems in their respective answer boxes.
 - If the answer box doesn't fit your answer, you may *continue* at the back of the **corresponding page** (**not** on a different page), but say so in the corresponding answer box.
 - Do not remove/replace the label identifying the question an answer box corresponds to.
 - Answers not following the above rules **will not be marked**.
 3. Place your bags in the front portion of the exam room, place all electronic communication devices in your bags, and make sure to set them to **silent mode**.
 4. Submit *all* answer sheets (even blank ones), your questionnaire, and all your scratch papers.
 - Make sure your name *and* student number are at the top of *each* answer sheet (even blank ones), front side.
 - Write your name, student number and section (or discussion class date/time) on the *upper-right-hand corner of the first page* of the questionnaire.
 - Write your name on every scratch paper you used.
 - You may write on the questionnaire. (They will not be considered part of your answers.)
 - Do **not** remove staple wires. (You may remove the removable fasteners.)
 5. Use a **black or blue pen** for all solutions, answers, and other written requirements. Pencils, any form of erasable ink, and red ink are **not** allowed. Answers not following this rule **will not be marked**.
 6. Write clearly and legibly. This includes writing in *complete sentences*, especially for proving and essay questions. If the answers are unclear, they may be *ignored*.
 7. For items with computations, show *complete* solutions, unless it is stated that solutions are not required. Answers with incomplete/lacking solutions will likely be given few to no points.
 8. Make sure that there is no missing page in this questionnaire.

Above all else, your goal is to *communicate*. After all, if a grader cannot understand your solution, they cannot give you any credit for it.

The exam will be scored over **1000 points**.

Exam Problems

[100♥ points] 1. Consider the following TypeScript object.

```
const xweet = {
  id: "1234567890123456789",
  text: "Para sa bawat kabataan na nais magka-diploma at makahanap ng trabaho; ...",
  createdAt: "2025-05-12T14:12:00.000Z",
  user: {
    id: "987654321",
    name: "Jam Arino",
    username: "jamarino",
    profileImageURL: "https://pbs.twimg.com/profile_images/1234567890/jamarino.jpg",
    verified: true,
  },
  metrics: {
    rexweetCount: 152,
    replyCount: 24,
    likeCount: 314,
    quoteCount: 8,
  },
  entities: {
    hashtags: [],
    urls: [],
    mentions: [],
  },
  possiblySensitive: false,
  lang: "en",
}
```

Consider the following type declarations:

```
type Type0 = {
  user: User
  metrics: XweetMetrics
  entities: Entities
}

type Type1 = {
  text: string
  possiblySensitive: boolean
  lang: string
}

type Type2 = {
  id: string
  text: string
  createdAt: Date
  possiblySensitive: boolean
}

type Type3 = {
  metrics: {
    mentions: Array<string>
  }
}

type Type4 = {
  metrics: {
    hashtags: [string][]
    urls: [string][]
    mentions: [string][]
  }
}

type Type5 = {
  lang: "en"
}
```

```
type Type6 = {
  id: string
  possiblySensitive: false
}

type Type7 = {
  id: number
  text: string
  createdAt: string
  possiblySensitive: boolean
  lang: string
}

type Type8 = {
  id: string
  text: string
  name: string
}

type Type9 = {
  metrics: number[]
}

type User = {
  username: string
  profileImageURL: string
  verified: boolean
}

type XweetMetrics = {
  rexweetCount: number
  quoteCount: number
}

type Entities = {
  hashtags: string[]
}
```

For each of the ten types `Type0`, `Type1`, ..., `Type9`, determine whether `xweet` is of that type, i.e., `xweet` can be used where that type is expected. Explicitly mark each one with either **Yes** or **No**.

Note that for this item, incorrect answers will give point deductions! A correct answer of either **Yes** or **No** gives 10 points, and an incorrect one gives -15 points. The maximum points you can get from this item is 100, and the minimum is -150 . A missing or blank answer is *not* considered incorrect.

- [115 points] 2. Provide an MVU-based implementation of a web app that contains three buttons, each containing its current counter value (initially 0). Whenever a button is clicked, its counter value is incremented by 1. Counter values that exceed 9 are turned into 0.

You must provide code for the `Model` and `Msg` types, for the `initModel` value, and for `update` and `view` functions.

An MVU template that you may opt to use is in Code Block A.

Code Block A Code for item 2

```
type Model = typeof Model.Type
const Model = S.Struct({
  // Fill this up
})

const initModel = Model.make({
  // Fill this up
})

type Msg = typeof Msg.Type
const Msg = S.Union(
  S.TaggedStruct("YourMessageNameHere", {}),
  // Fill this up
)
const [MsgA, ...] = Msg.members

const update = (msg: Msg, model: Model) =>
  Match.value(...).pipe(
    Match.tag("...", () =>
      Model.make({
        ...
      }),
    ),
    // Fill this up
    Match.exhaustive,
  )

const view = (model: Model, dispatch: (msg: Msg) => void) =>
  h("div", [
    h(
      "button",
      {
        on: {
          click: () => dispatch(YourMessageNameHere.make({ ... })),
        },
      },
      `Your button text here`,
    ),
  ])
  ])
```

No need to add `import` statements or the `startSimple` call. You may use regular JavaScript/-TypeScript instead of Effect, but you must ensure that no mutation is performed.

Syntax errors will be tolerated. You may also use brace functions instead of arrow functions.

3. For the following subitems, consider the `callback` function in Code Block B which is executed whenever a button on a web page is pressed:

- [80♥ points] (a) Explain the reason why the browser freezes for a moment when the button is pressed before showing just the final result on screen.

Code Block B Code for item 3

```
function callback() {
  const idxElem = document.getElementById("idx")!
  const sumElem = document.getElementById("sum")!
  let sum = 0;

  for (let i = 1; i <= 5000000; i++) {
    idxElem.innerText = `i: ${i}`
    sum += i
    sumElem.innerText = `sum: ${sum}`
  }
}
```

[60 points]

- (b) Explain how the code below results in the text on screen being updated with the values of sum and i at a very high frequency.

```
async function callback() {
  const idxElem = document.getElementById("idx")!
  const sumElem = document.getElementById("sum")!
  let sum = 0;

  for (let i = 1; i <= 5000000; i++) {
    idxElem.innerText = `i: ${i}`
    sum += i
    sumElem.innerText = `sum: ${sum}`

    if (i % 107 == 0) {
      await new Promise((resolve) => setTimeout(resolve, 0))
    }
  }
}
```

[50 points]

- (c) Rewrite the async version of callback as a non-async function that returns a **Promise**. (Hint: One approach is to turn the for loop into a recursive function defined inside callback, then determine whether the current recursive “iteration” should temporarily yield control)

4. Consider two implementations of fetching data from <https://randomuser.me/api/> via fetch:

Program 1:

```
const fetchAll1 = () => {
  const p1 = new Promise((resolve) => {
    console.log(`Sootopolis`)
    fetch("https://randomuser.me/api/").then((x) => {
      console.log(`Verdanturf`)
      resolve(x)
    })
    console.log(`Mauville`)
  })

  const p2 = new Promise((resolve) => {
    console.log(`Olivine`)
    fetch("https://randomuser.me/api/").then((x) => {
      console.log(`Azalea`)
      resolve(x)
    })
    console.log(`Ecruteak`)
  })

  const p3 = new Promise((resolve) => {
    console.log(`Cinnabar`)
    fetch("https://randomuser.me/api/").then((x) => {
      console.log(`Celadon`)
      resolve(x)
    })
    console.log(`Vermilion`)
  })

  return p1.then((a) => p2.then((b) => p3.then((c) => [a, b, c])))
}
```

Program 2:

```
const fetchAll2 = async () => {
  const urls = [
    "https://randomuser.me/api/",
    "https://randomuser.me/api/",
    "https://randomuser.me/api/",
  ]

  const res: Response[] = []

  for (let i = 0; i < urls.length; i++) {
    console.log(`Nanahama ${i}`)
    res.push(await fetch(urls[i]))
    console.log(`Kawakami ${i}`)
  }

  return res
}
```

Assume that the first time fetch is called by a program will take 5 seconds, the second time will take 7 seconds, and the third will take 3 seconds. Assume as well that all lines other than fetch take negligible time to execute.

[55 points]

(a) Determine the output of **Program 1**. For each line in the output, explain how the flow of the program results in the printing of the said line.

[55 points]

(b) Determine the output of **Program 2**. For each line in the output, explain how the flow of the program results in the printing of the said line.

5. Consider two similar TypeScript programs:

Program 1:

```
const nums = [10, 20, 30]

// (() => void) means zero-argument function with discarded return
// (() => void)[] means array of (() => void)
const fns: (() => void)[] = []

let num = 0
while (num < 3) {
  function f() {
    console.log(num)
  }

  fns.push(f)
  num++
}

for (const fn of fns) {
  fn()
}
```

Program 2:

```
const nums = [10, 20, 30]

const fns: (() => void)[] = []

for (let num = 0; num < 3; num++) {
  function f() {
    console.log(num)
  }

  fns.push(f)
}

for (const fn of fns) {
  fn()
}
```

[70♥ points]

(a) Identify the output of each program above.

[55 points]

(b) Illustrate using execution frames and closures why both programs have different outputs.

6. Consider code snippets of two similar TypeScript programs:

Snippet 1:

```
type Result = {
  readonly words: string
  readonly factors: number[]
}

function doSomething(result: Result) { ... }

function f(n: number) {
  const result: Result = computeWordAndFactors(n)

  doSomething(result)

  if (result.factors.length === 2) {
    // 100 lines here; has assignment
    // statements, but no function calls
    // ...
  }

  return result
}
```

Snippet 2:

```
type Result = {
  readonly words: string
  readonly factors: readonly number[]
}

function doSomething(result: Result) { ... }

function f(n: number) {
  let result: Result = computeWordAndFactors(n)

  doSomething(result)

  if (result.factors.length === 2) {
    // 100 lines here; has assignment
    // statements, but no function calls
    // ...
  }

  return result
}
```

Assume that type guarantee-circumventing constructs (e.g., type assertions, type casting) are not present in the codebase. Note that `doSomething` and `computeWordAndFactors` being incomplete or unspecified is intended. `f` and `computeWordAndFactors` are expected to always return the same values.

Suppose that testing reveals that both `f` implementations fail a subset of unit tests despite `computeWordAndFactors` being implemented correctly. To ease debugging effort, you would like to rule out possibilities of which parts may be affecting the value returned by `f`.

[70 points]

(a) For each snippet, identify whether it is possible for the portion below to cause `f` to behave incorrectly and justify your claim. If yes, include a concrete example along with your explanation:

- `doSomething`
- The 100-line `if-else` statement with no function calls

[50 points]

(b) Consider a third snippet with `f` that passes all unit tests:

Snippet 3:

```
type Result = {
  readonly words: string
  readonly factors: readonly number[]
}

function doSomething(result: Result) { ... }

const tap = <T>(func: (x: T) => void) => (x: T): T => {
  func(x)
  return x
}

const f = (n: number): Result =>
  pipe(
    computeWordAndFactors(n),
    tap((result) => { // First anonymous function
      doSomething(result)
    }),
    tap((result) => { // Second anonymous function
      if (result.factors.length === 2) {
        // 100 lines here; has assignment
        // statements, but no function calls
        // ...
      }
    })
  ),
)
```

Provide separate explanations for each anonymous function above as to why they are unable to make `f` return a value that is not equal to the value returned by `computeWordAndFactors(n)`.

7. For the CS 12 24.2 project, groups can be comprised of 1–4 students.

Code Block C Code for item 7

```
@dataclass(frozen=True)
class ProjectGroup:
    group_name: str
    member1: str | None
    member2: str | None
    member3: str | None
    member4: str | None
```

- [60 points] (a) Give two distinct reasons why the data representation in Code Block C used to represent a group can be problematic/inconvenient and provide a concrete example for each reason. Ignore mutability/immaturity concerns and the possibility of having duplicate members in the same group and across groups.

- [75 points] (b) Provide an improved version of `ProjectGroup` in **Python** that addresses both reasons given in (a) and explain how it addresses them.

8. Consider the C code snippet in Code Block D.

- [50♥ points] (a) Identify the output of the first two `printf` calls.
- [45 points] (b) Identify the output of the remaining (four) `printf` calls, and explain them.
- [50♥ points] (c) If `p32` was instead initialized as `arr32 + 1.5`, determine what would happen, and explain why.

- [60 points] 9. Precisely describe the value returned by the function `f` in Code Block E in terms of its `nums` parameter.

10. `Map<K, V>` is the JavaScript/TypeScript version of `dict[K, V]` in Python. Its `.get(key: K)` method returns the value associated with `key` if it is present in the `Map`, or `undefined` otherwise. The `increment` function in Code Block F mutates its `Map<string, number>` argument by incrementing the count associated with the `name` argument taken as a key, or associate it with 1 if it is not present in the `Map`:

Code Block D Code for item 8

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

int main() {
    int32_t arr32[] = {10, 20, 30, 40, 50};
    int64_t arr64[] = {100, 200, 300, 400, 500};

    int32_t *p32 = arr32 + 3;
    int64_t *p64 = arr64 + 2;

    printf("%" PRIi32 "\n", *p32); // "%" PRIi32 "\n" becomes "%d\n" in Linux
    printf("%" PRIi64 "\n", *p64); // "%d" PRIi64 "\n" becomes "%ld\n" in Linux

    printf("%ld\n", p32 - arr32);
    printf("%ld\n", p64 - arr64);

    printf("%ld\n", (char*)p32 - (char*)arr32);
    printf("%ld\n", (char*)p64 - (char*)arr64);

    return 0;
}
```

Code Block E Code for item 9

```
const f = (nums: number[]) =>
  nums.length === 0 ?
    null
  : nums.reduce(
    ([a, b], cur) =>
      [cur < a ? cur : a, cur > b ? cur : b] as [number, number],
    [nums[0], nums[0]],
  )
```

Code Block F Code for item 10

```
function increment(counter: Map<string, number>, name: string): void {
  const cur = counter.get(name) ?? 0;

  counter.set(name, cur + 1);
}

function caller(tally: Map<string, number>) {
  increment(tally, "Daryll")
  increment(tally, "Payton")
  increment(tally, "Payton")
  increment(tally, "Justin")
  increment(tally, "Daryll")

  return tally
}
```


[80 points]

- (a) Assuming `HashMap` is already imported from the `effect` package, refactor both `increment` and `caller` such that the immutable `HashMap.HashMap<K, V>` is used instead of the built-in `Map<K, V>`.

Ensure that the `HashMap.HashMap` returned by the new `increment` is equivalent to the `Map` returned by the old `increment`. While you may change their return types, you may not change their parameters other than changing `Map<string, number>` annotations to `HashMap.HashMap<string, number>`. You may use `pipe` if necessary. You may refer to [Appendix A](#) for documentation on possibly useful `HashMap` functions.

[50 points]

- (b) Explain why there is no need for the original `caller` implementation to return `tally`, but is necessary for the rewritten `caller` in (a). Your reasoning should also apply to `increment` (same fundamental issue).

Code Block G Code for item 11

```
11. #include <stdio.h>
    #include <stdlib.h>

    int **make_three_heap_blocks() {
        unsigned int n1, n2, n3;
        int *p1, *p2, *p3;
        int *pointers[3] = {};

        if (scanf("%u", &n1) == 0 || n1 == 0) {
            printf("Invalid input.\n");
            return NULL;
        }
        p1 = malloc(n1);

        if (scanf("%u", &n2) == 0 || n2 == 0) {
            printf("Invalid input.\n");
            return NULL;
        }
        p2 = malloc(n2);

        if (scanf("%u", &n3) == 0 || n3 == 0) {
            printf("Invalid input.\n");
            return NULL;
        }
        p3 = malloc(n3);

        pointers[0] = p1;
        pointers[1] = p2;
        pointers[2] = p3;

        return pointers;
    }
```

Consider Code Block G. The function `make_three_heap_blocks` is supposed to return three pointers to three separate heap allocations. The user is asked for three positive integers, each corresponding to the number of unsigned 32-bit integers an allocation can accommodate. Should any of the user inputs be invalid, `NULL` should be returned.

[60♥ points]

- (a) Rewrite `make_three_heap_blocks` without deleting any existing lines and without changing what is returned such that all instances of memory leaks are removed.

[55 points]

- (b) Rewrite `make_three_heap_blocks` (with deletions allowed) such that all Heisenbug-causing behaviors are fixed while ensuring that `make_three_heap_blocks` does what it is intended to do and that no memory is leaked.

[65 points] 12. The `ModelCmd<Model, Msg>` type of `cs12252-mvu` was initially defined as a single object:

```
type ModelCmd<Model, Msg> = {
    model: Model
    cmd: Cmd<Msg>
}
```

The current version of `cs12252-mvu` has `ModelCmd<Model, Msg>` defined using a type union:

```
type ModelCmd<Model, Msg> =  
  | Model  
  | {  
    model: Model  
    cmd: Cmd<Msg>  
  }
```

Explain what advantage the current version of `ModelCmd<Model, Msg>` has over the original version in the context of MVU.

Bonus

[4♥ points] **13.** What is the capital of Assyria?

合格しますように！！

A Some HashMap methods

Here are some HashMap methods you might need.

- `HashMap.has`
 - `arg1: HashMap.HashMap<K, V>`
 - `arg2: K`
 - Returns whether `arg2` is in `arg1`
- `HashMap.set`
 - `arg1: HashMap.HashMap<K, V>`
 - `arg2: K`
 - `arg3: V`
 - Returns a copy of `arg1` in which `arg2` is associated with `arg3`
- `HashMap.unsafeGet`
 - `arg1: HashMap.HashMap<K, V>`
 - `arg2: K`
 - Returns the value in `arg1` associated with `arg2` if it exists; raises an error otherwise
- `HashMap.modify`
 - `arg1: HashMap.HashMap<K, V>`
 - `arg2: K`
 - `arg3: (v: V) => V`
 - If `arg2` is in `arg1`, do `HashMap.set(arg1, arg3(HashMap.unsafeGet(arg1, arg2)))`